

Frontend Performance Optimization

Interview questions with detailed answers, plus the follow-up (counter) questions interviewers actually ask — and how to answer them.

Contents

1. Metrics & Measurement (Core Web Vitals)
2. Network & Loading
3. Caching & Service Workers
4. JavaScript & Bundling
5. Rendering & the Browser Pipeline
6. Images, Fonts & Assets
7. React-Specific Optimization
8. Memory, Long Lists & Runtime
9. Rendering Architecture (CSR/SSR/SSG/ISR)
10. Rapid-Fire Round

1. Metrics & Measurement (Core Web Vitals)

Q1. What are Core Web Vitals and what does each one actually measure?

Three user-centric metrics Google uses as ranking signals, each with a "good" threshold measured at the **75th percentile of real users**:

Metric	Measures	Good	Main causes of failure
LCP (Largest Contentful Paint)	Time until the largest above-the-fold element (image, video poster, big text block) is painted — perceived load speed	< 2.5s	Slow TTFB, render-blocking CSS/JS, unoptimized hero image, client-side-only rendering
INP (Interaction to Next Paint)	Replaced FID in March 2024. Worst-ish (98th pct) latency from any user interaction to the next frame painted — full responsiveness, not just first input	< 200ms	Long tasks, heavy event handlers, large synchronous re-renders, hydration
CLS (Cumulative Layout Shift)	Sum of unexpected layout shift scores (impact fraction × distance fraction) over the page's life	< 0.1	Images/iframes/ads without dimensions, late-injected banners, FOIT/FOUT font swaps

Supporting (non-CWV) metrics: **TTFB** (server + network), **FCP** (first pixel of content), **TBT** (lab proxy for INP — total time of long tasks beyond 50ms between FCP and TTI).

Counter-Q: Your Lighthouse score is 98 but CrUX field data says LCP is failing. How is that possible and what do you do?

A: Lighthouse is *lab* data — a single simulated run, throttled Moto G, cold cache, one geography, no logged-in state, no third-party consent banners firing. Field data (CrUX/RUM) is the 75th percentile of *real* users across devices, networks, geographies, and warm/cold cache. Divergence usually comes from: (a) real users hitting a slower region with no CDN POP, (b) LCP element differing for logged-in users, (c) A/B test or personalization scripts blocking render, (d) mobile traffic dominating while you tested desktop. Fix: install a RUM library (web-vitals.js) and ship LCP with `attribution` build — it reports the LCP element selector, TTFB / resource-load-delay / load-duration / render-delay sub-parts. Segment by device, country, and route. Optimize the sub-part that dominates, not the aggregate number.

Q2. How would you set up performance monitoring for a production app?

Two layers:

- **Lab / synthetic (pre-merge):** Lighthouse CI in the pipeline with performance budgets in `lighthouseci.json` — fail the build if LCP regresses or bundle exceeds N KB. Deterministic, catches regressions before users do.
- **Field / RUM (post-deploy):** the `web-vitals` library reporting to your analytics endpoint via `navigator.sendBeacon` on `visibilitychange` (not `unload` — unreliable on mobile).

```
import { onLCP, onINP, onCLS, onTTFB } from 'web-vitals/attribution';

const queue = new Set();
function report(metric) {
  queue.add({ ...metric, route: location.pathname, conn: navigator.connection?.effectiveType });
}
[onLCP, onINP, onCLS, onTTFB].forEach(fn => fn(report));

addEventListener('visibilitychange', () => {
  if (document.visibilityState === 'hidden' && queue.size) {
    navigator.sendBeacon('/rum', JSON.stringify([...queue]));
    queue.clear();
  }
});
```

Also use the `PerformanceObserver` API directly for `longtask`, `resource`, and `element` timing. Always report **p75/p95, never the mean** — averages hide the tail where users actually churn.

Counter-Q: Your RUM script itself costs performance. How do you keep it cheap?

A: Keep it <2KB (web-vitals is ~1.5KB gzipped), load it with `defer` or inline it, batch into a single beacon at page-hide instead of one request per metric, sample high-traffic routes (e.g. 10% of sessions still gives statistically valid p75 at scale), and do all aggregation server-side. Never send on every interaction.

Q3. Explain the critical rendering path.

The sequence from bytes to pixels:

1. **HTML → DOM:** parser tokenizes and builds the DOM tree incrementally.
2. **CSS → CSSOM:** CSS is **render-blocking** — the browser cannot paint until the CSSOM is complete, because a later rule could restyle everything.
3. **DOM + CSSOM → Render Tree:** only visible nodes (`display:none` excluded; `visibility:hidden` included).
4. **Layout / Reflow:** compute geometry (position + size) for each box.
5. **Paint:** fill in pixels into layers.
6. **Composite:** GPU assembles layers in the right order to the screen.

Key: a synchronous `<script>` is **parser-blocking** — it halts DOM construction, and because scripts can read computed styles, it also waits for pending CSS. That's why CSS goes in `<head>` and scripts get `defer`.

To optimize: minimize critical resources, minimize critical bytes (minify, compress with Brotli), minimize round trips (inline critical CSS, preload key assets).

Counter-Q: Difference between `async` and `defer`, and when would you use neither?

A: Both download in parallel with parsing. `async` executes *the moment it arrives*, pausing the parser, and order is not guaranteed — right for independent scripts with no DOM dependency (analytics, error trackers). `defer` executes after HTML parsing completes, just before `DOMContentLoaded`, and **preserves document order** — the default for app code. `type="module"` is deferred by default. Use neither (plain blocking) only for things that must run before first paint — an anti-flicker snippet for A/B testing, or a small inline config/feature-flag object. Even then, inline it rather than making a network request.

2. Network & Loading

Q4. A page takes 8 seconds to load. Walk me through your debugging process.

Systematic, top-down — never guess:

1. **Split the 8s** in DevTools Network → the waterfall. Is it TTFB (server/backend/CDN miss), download (payload size), or processing (main thread)?
2. **High TTFB (>600ms)**: backend query, no CDN edge caching, cold serverless start, redirect chains. Check `Server-Timing` headers.
3. **Big payload**: check Coverage tab for unused CSS/JS %, run bundle analyzer, verify Brotli/gzip is on, check image formats and dimensions.
4. **Waterfall staircase** (requests starting sequentially): request chains — CSS importing CSS, JS fetching JS, fonts discovered only after CSS parses. Fix with `preload` / `preconnect` and flattening imports.
5. **Main thread red in Performance tab**: long tasks, expensive hydration, huge JS parse/compile. Look at the flame chart bottom-up.
6. **Third parties**: sort waterfall by domain. Tag managers, chat widgets, and ad scripts are routinely 40–60% of the problem.
7. Re-measure after *each* change so you attribute the win correctly.

Counter-Q: TTFB is 2s and the backend team says their p99 is 80ms. Who's right?

A: Both can be. TTFB as the browser sees it = DNS + TCP + TLS + request travel + server processing + first byte travel. Their 80ms only covers server processing. The gap is usually: no CDN so every user crosses an ocean, a 301 redirect chain (`http→https→www` = two extra full round trips), DNS resolution for a cold domain, or TLS negotiation. Verify with the Timing tab breakdown in DevTools, which itemizes exactly these. Fixes: CDN with edge POPs, eliminate redirects (canonical URL in links), HTTP/2 or /3, `preconnect` to critical third-party origins, and `Early Hints (103)` if your CDN supports it.

Q5. Explain preload, prefetch, preconnect, dns-prefetch, and prerender.

Hint	What it does	Use for
dns-prefetch	Resolves DNS only. Cheapest, widest support.	Domains you'll <i>probably</i> use later
preconnect	DNS + TCP + TLS handshake up front (saves ~100–300ms)	Critical third-party origins — font CDN, API host. Limit to 3–4; each holds a socket open.
preload	High-priority fetch of a resource needed on this page . Requires <code>as</code> attribute.	LCP hero image, self-hosted fonts, late-discovered critical CSS/JS
prefetch	Low-priority idle fetch for a future navigation	The next page in a known flow (product → checkout)
prerender / Speculation Rules API	Renders the whole next page invisibly in the background	Very high-confidence next navigation only — expensive in CPU/memory/data

```
<link rel="preconnect" href="https://api.example.com" crossorigin>
<link rel="preload" as="image" href="/hero.avif" fetchpriority="high">
<link rel="preload" as="font" type="font/woff2" href="/inter.woff2" crossorigin>

<script type="speculationrules">
{ "prerender": [{ "where": { "href_matches": "/checkout" }, "eagerness": "moderate" }] }
</script>
```

Counter-Q: Why does preload on a font need crossorigin even when the font is on my own domain?

A: Fonts are always fetched in CORS/anonymous mode by the CSS font loading spec, regardless of origin. If the `preload` is issued without `crossorigin`, the browser stores it under a different cache key than the later CORS-mode request — so the font is downloaded **twice** and the preload actively hurt you. This is the single most common preload bug. DevTools warns: "resource was preloaded but not used within a few seconds."

Q6. How do HTTP/2 and HTTP/3 change frontend optimization practice?

HTTP/1.1 allowed ~6 parallel connections per origin, so we did *domain sharding*, *sprite sheets*, and *bundle everything into one file* to dodge the limit.

HTTP/2 added multiplexing (many streams over one TCP connection), header compression (HPACK), and stream prioritization. Consequences: sharding is now **harmful** (extra handshakes, defeats multiplexing), sprites are mostly obsolete, and **granular chunks are better than one megabundle** — a change to one route invalidates only that chunk's cache. Server Push was part of H/2 but is deprecated/removed (Chrome dropped it) — `preload` + `103 Early Hints` replaced it.

HTTP/3 runs over QUIC/UDP, which eliminates **TCP head-of-line blocking** — under packet loss, H/2 stalls all streams on one lost packet; H/3 stalls only the affected stream. It also does 1-RTT (or 0-RTT resumption) connection setup since TLS is baked in. Biggest wins on lossy mobile networks.

Counter-Q: So with HTTP/2 should I just ship 500 tiny modules unbundled?

A: No — multiplexing removes the *connection* cost, not the *per-request* cost. Each file still carries header overhead, a separate cache entry, and its own compression context, so gzip/Brotli ratios collapse on tiny files (shared dictionary is lost). Plus the browser must resolve a deep module graph, which serializes discovery. The practical sweet spot is roughly **20–50KB chunks**: split by route and by vendor-vs-app, extract genuinely shared modules, and let the bundler merge chunks below a minimum size threshold (`maxInitialRequests` / `minSize` in Webpack's `splitChunks`).

3. Caching & Service Workers

Q7. Explain browser caching headers and design a caching strategy for an SPA.

- `Cache-Control: max-age=N` — fresh for N seconds, served from disk with **zero network**.
- `immutable` — don't even revalidate on reload. Only for content-hashed files.
- `no-cache` — cache it, but **always revalidate** before use (not the same as no-store).
- `no-store` — never write to disk. For sensitive/personalized responses.
- `ETag` / `Last-Modified` → conditional request → `304 Not Modified`: saves bandwidth but still costs a round trip.
- `stale-while-revalidate=N` — serve stale instantly, refresh in background. Great for API responses and at the CDN.

SPA strategy:

```
index.html      → Cache-Control: no-cache          # always revalidate, cheap 304
/assets/*.js|css → max-age=31536000, immutable
/images/*       → max-age=604800, stale-while-revalidate=86400
/api/*          → no-store (or short max-age + SWR for public data)
```

The rule: **the HTML entry point is never cached hard; everything it references is content-hashed and cached forever**. A deploy changes the hashes, the fresh HTML points at new URLs, and cache invalidation solves itself.

Counter-Q: You deployed, users get a white screen and "Failed to fetch dynamically imported module". Why?

A: Classic stale-chunk problem. A user had the old `index.html` loaded when you deployed; they then lazy-load a route, requesting `Home.a1b2.js`, which no longer exists on the CDN — 404 → the dynamic import rejects. Fixes, layered: (1) **keep old chunks on the CDN** for a deploy window (a few versions / 24–48h) instead of purging — cheapest and most effective; (2) catch the import failure and force `location.reload()` once, guarded by a `sessionStorage` flag to avoid a reload loop; (3) poll a `/version.json` or use a service-worker `updatefound` event to show a "New version available — Refresh" toast; (4) with a service worker, precache the full asset manifest so the old shell keeps working offline.

Q8. What caching strategies would you implement in a Service Worker?

The SW is a network proxy on a separate thread, so you choose per-request-type:

- **Cache First** — check cache, fall back to network. For hashed static assets and fonts. Fastest, risk of staleness (none if hashed).
- **Network First** — try network, fall back to cache. For the HTML shell and fresh-critical API data. Adds offline resilience without staleness.
- **Stale-While-Revalidate** — return cache immediately, fetch and update in background. Best UX/freshness tradeoff for avatars, non-critical API data, CSS.
- **Network Only / Cache Only** — for analytics beacons and precached shells respectively.

```
self.addEventListener('fetch', (e) => {
  const { request } = e;
  if (request.mode === 'navigate') {
    e.respondWith(fetch(request).catch(() => caches.match('/offline.html')));
  } else if (/\. (js|css|woff2)$/.test(new URL(request.url).pathname)) {
    e.respondWith(caches.match(request).then(c => c || fetch(request)));
  }
});
```

In practice use **Workbox** rather than hand-rolling — it handles precache manifests, cache expiration, and versioning correctly.

Counter-Q: A bad service worker shipped and it's caching a broken app. Users are stuck. How do you recover?

A: This is the "SW is a footgun" scenario — the SW can outlive your deploy. Recovery: ship a new SW that in its `install` handler calls `self.skipWaiting()`, in `activate` calls `caches.keys()` and deletes everything, then `self.clients.claim()` and reloads clients. If the SW file itself is cached, note browsers cap SW script cache at 24h and honor `Cache-Control`, so **always serve `sw.js` with `no-cache`**. Nuclear option: deploy a "kill switch" SW that calls `self.registration.unregister()`. Prevention: never precache the HTML shell without a version check, and stage SW changes behind a flag.

4. JavaScript & Bundling

Q9. Your JS bundle is 3MB. How do you reduce it?

Measure first — `webpack-bundle-analyzer` / `rollup-plugin-visualizer` / `vite-bundle-visualizer` — then:

1. **Code splitting by route:** `React.lazy(() => import('./Dashboard'))` + `<Suspense>`. Usually the single biggest win.
2. **Component-level splitting:** modals, charts, rich-text editors, date pickers — load on interaction, not on page load.
3. **Tree shaking:** requires ESM (`import` / `export`), `"sideEffects": false` in `package.json`, and *not* transpiling modules to CJS (`modules: false` in `babel-preset-env`).
4. **Replace heavy dependencies:** `moment` (~70KB) → `date-fns/dayjs` (~2–7KB); `lodash` → `lodash-es` with named imports; full icon packs → individual icon imports.
5. **Polyfill diet:** ship modern ESM to modern browsers via `<script type="module">` + `nomodule` fallback, or drop legacy targets entirely; use `core-js` with `useBuiltIns: 'usage'`.
6. **Dedupe:** multiple React copies or two date libraries — check with `npm ls` and bundler duplicate detection.
7. **Compression:** Brotli level 11 for static assets (~15–20% better than gzip).
8. **Guardrails:** add a size budget to CI (`bundlesize` , `size-limit`) so this never recurs.

Counter-Q: You imported `{ debounce }` from `'lodash'` and the whole library still shipped. Why?

A: The main `lodash` package is CommonJS. Tree shaking is a *static* analysis over ES modules; CJS `require` / `module.exports` are dynamic, so the bundler can't safely prove what's unused and keeps everything. Fixes: use `lodash-es` (ESM build), or import the single module path `import debounce from 'lodash/debounce'`, or use `babel-plugin-lodash`. Same class of bug: a package marked with side effects, or Babel transpiling your ESM to CJS before Webpack sees it. For a 3-line utility like `debounce`, honestly just write it yourself.

Q10. Explain code splitting and how you decide the split points.

Code splitting breaks the bundle into chunks loaded on demand, via the dynamic `import()` which returns a Promise and is a signal to the bundler to create a chunk boundary.

Split points, in priority order:

- **Route level** — the default. Users on `/login` shouldn't download the admin dashboard.
- **Interaction level** — modal, video player, comment editor, map. Preload on `mouseenter` / `focus` so the click feels instant.
- **Viewport level** — below-the-fold widgets, via `IntersectionObserver`.
- **Vendor split** — long-lived third-party code in its own chunk so app deploys don't bust it. (Less valuable with modern content hashing than it used to be.)
- **Conditional** — admin-only code, locale bundles, polyfills for old browsers.

```
const Chart = lazy(() => import(/* webpackChunkName: "chart" */ './Chart'));

// warm the chunk before it's needed
<button onMouseEnter={() => import('./Chart')} onClick={open}>Show chart</button>
```

Counter-Q: You split aggressively and the app got slower. What happened?

A: Over-splitting creates a **request waterfall**: the browser must download and execute chunk A to discover it needs chunk B, which needs C — three serialized round trips instead of one parallel fetch. Symptoms: spinner cascades, poor INP on navigation, and worse compression ratios on many tiny files. Remedies: set `minSize` / `maxInitialRequests` so the bundler merges trivial chunks, use `<link rel="modulepreload">` so the browser fetches the graph in parallel instead of discovering it lazily, prefetch likely-next chunks during idle, and don't split anything needed for the initial paint. Also: every `<Suspense>` boundary that flashes a spinner is a CLS risk — reserve the space.

Q11. Debouncing vs throttling — implement both and say when each applies.

Debounce: run the function only after N ms of *silence*. Collapses a burst into one final call. **Throttle:** run at most once per N ms during a burst. Guarantees a steady rate.

```
function debounce(fn, wait) {
  let t;
  const debounced = (...args) => {
    clearTimeout(t);
    t = setTimeout(() => fn(...args), wait);
  };
  debounced.cancel = () => clearTimeout(t);
  return debounced;
}

function throttle(fn, limit) {
  let last = 0, timer;
  return (...args) => {
    const now = Date.now();
    const remaining = limit - (now - last);
    if (remaining <= 0) { last = now; fn(...args); }
    else { // trailing call so the final event isn't dropped
      clearTimeout(timer);
      timer = setTimeout(() => { last = Date.now(); fn(...args); }, remaining);
    }
  };
}
```

Debounce: search-as-you-type (300ms), autosave, resize-end recalculation, form validation.

Throttle: scroll position tracking, mousemove/drag, infinite-scroll triggers, analytics events, rate-limited API polling.

Counter-Q: For scroll and resize, is throttle even the right tool anymore?

A: Usually not. For *visibility* ("is this element on screen?") use `IntersectionObserver` — it runs off the main thread and fires only on state change, so it beats any throttled scroll handler for lazy loading, infinite scroll, and sticky headers. For *element size* use `ResizeObserver`, which also catches non-window-driven size changes a resize handler misses entirely. If you genuinely need per-frame scroll work (parallax), use `requestAnimationFrame` coalescing rather than a time-based throttle, so you run exactly once per frame in sync with paint. And always pass `{ passive: true }` on scroll/touch listeners so the browser doesn't wait to see if you'll call `preventDefault()`.

Q12. What is a "long task" and how do you break one up?

Any main-thread task >50ms. During it the thread can't respond to input, run animations, or paint — it is the direct cause of bad INP/TBT. Strategies:

- **Yield to the main thread** between chunks of work so the browser can service pending input:

```
function yieldToMain() {
  if ('scheduler' in globalThis && scheduler.yield) return scheduler.yield();
  return new Promise(r => setTimeout(r, 0));
}

async function processAll(items) {
  for (const item of items) {
    doWork(item);
    if (performance.now() - start > 50) { await yieldToMain(); start = performance.now(); }
  }
}
```

- **Web Worker** for genuinely CPU-bound work (parsing large JSON/CSV, image processing, crypto, diffing) — it's off the main thread entirely. Use `comlink` to avoid `postMessage` boilerplate; transfer `ArrayBuffer`s instead of copying.
- **Prioritize** with `scheduler.postTask({ priority: 'background' })` for non-urgent work, or `requestIdleCallback`.
- **Virtualize / paginate** so you never process 10k items at once.
- **Defer third-party scripts** — often the actual source of the long task.

Counter-Q: Why not just put everything in a Web Worker?

A: Workers have no DOM access, so anything touching the UI must come back to the main thread. Communication is `postMessage` with structured cloning — for large objects the serialization cost can exceed the work you offloaded (transferables and `SharedArrayBuffer` mitigate this but constrain your data shapes). Each worker also costs memory and startup time. So: worker for *pure computation on serializable data*; main thread with yielding for *DOM-adjacent* work. Measure the `postMessage` overhead before committing.

5. Rendering & the Browser Pipeline

Q13. What is layout thrashing and how do you fix it?

Forced synchronous layout: you write to the DOM (invalidating layout), then immediately read a geometry property, forcing the browser to recompute layout *synchronously* mid-JavaScript instead of batching it before the next paint. In a loop this is $O(n)$ reflows.

```
// BAD – read/write interleaved, reflow per iteration
boxes.forEach(box => { box.style.width = box.offsetWidth + 10 + 'px'; });

// GOOD – batch all reads, then all writes
const widths = boxes.map(b => b.offsetWidth);           // read phase
boxes.forEach((b, i) => b.style.width = widths[i] + 10 + 'px'); // write phase
```

Layout-triggering reads: `offsetTop/Left/Width/Height`, `clientWidth/Height`, `scrollTop/Height`, `getBoundingClientRect()`, `getComputedStyle()`, `focus()`.

Other tools: `requestAnimationFrame` to schedule writes, `FastDom` to enforce read/write batching, and CSS `contain: layout style paint` or `content-visibility: auto` to scope layout work to a subtree so an off-screen change can't reflow the whole page.

Counter-Q: Which CSS properties can animate without triggering layout or paint, and why does that matter?

A: Only `transform` and `opacity` (and `filter`, mostly) can be handled entirely by the **compositor thread** on the GPU. Animating `left/top/width/height/margin` triggers layout → paint → composite every *frame*, on the main thread — so the animation stutters whenever JS is busy. Animating `background-color` or `box-shadow` skips layout but still repaints. With `transform / opacity`, the element is promoted to its own layer and the GPU just re-composites it, so it stays 60fps even under main-thread load. Promote deliberately with `will-change: transform`, but **sparingly** — each layer costs GPU memory, and promoting hundreds of elements causes "layer explosion" that is slower than not promoting at all. Remove `will-change` after the animation ends.

Q14. How do you achieve a consistent 60fps?

60fps = a frame every **16.7ms**, and the browser needs part of that for style/layout/paint/composite — so your JS budget is roughly **8–10ms per frame**. Rules:

- Animate with `transform / opacity` only; prefer CSS animations/WAAPI over JS (they can run off main thread).
- Use `requestAnimationFrame` for JS-driven animation — never `setInterval`, which drifts and doesn't align with the paint cycle.
- Keep the main thread free: no long tasks, virtualize long lists, debounce/throttle handlers, `passive` listeners.
- Reduce paint area — check "Paint flashing" and "Layer borders" in DevTools Rendering panel.
- Avoid expensive styles in animation: large `box-shadow`, `filter: blur()`, `backdrop-filter` on big surfaces.
- Use `content-visibility: auto` + `contain-intrinsic-size` to skip rendering off-screen sections entirely.

Counter-Q: The animation is smooth on your MacBook and janky on a mid-range Android. What's different and what do you do?

A: Mid-range Android has ~4–6× slower single-core CPU, far less GPU memory, thermal throttling, and often a 90/120Hz display (a 11ms or 8ms budget, not 16.7ms). Actions: (1) always profile with CPU throttling 4×/6× in DevTools, or on a real device via remote debugging; (2) reduce layer count and texture sizes — GPU memory is the usual Android bottleneck; (3) respect `prefers-reduced-motion` and consider degrading animations on low-end devices detected via `navigator.hardwareConcurrency / deviceMemory / navigator.connection.saveData`; (4) verify you're not accidentally repainting a full-screen layer each frame. The general principle: **your dev machine is the 99th percentile of your users' hardware** — never treat it as the baseline.

6. Images, Fonts & Assets

Q15. How do you optimize images? (Usually the single biggest payload win.)

- **Format:** AVIF (best compression, ~50% smaller than JPEG) → WebP (universal now) → JPEG/PNG fallback. SVG for icons/logos. Serve via `<picture>` with `<source type>`.
- **Responsive sizing:** `srcset` + `sizes` so a phone doesn't download a 2000px-wide image.
- **Lazy load** below-the-fold: `loading="lazy"` (native, free). **Never lazy-load the LCP image** — it delays the very metric you're optimizing.
- **Prioritize the hero:** `fetchpriority="high"` + `preload`.
- **Prevent CLS:** always set `width` / `height` (or `aspect-ratio`) so the browser reserves the box before the bytes arrive.
- **Decode:** `decoding="async"` keeps image decode off the critical path.
- **Pipeline:** use an image CDN (Cloudinary, imgix, Next/Image, Vercel) to transform on the fly by device and DPR rather than committing variants to git.

```
<picture>
  <source type="image/avif" srcset="/hero-480.avif 480w, /hero-1200.avif 1200w" sizes="(max-w
  <source type="image/webp" srcset="/hero-480.webp 480w, /hero-1200.webp 1200w" sizes="...">
  
</picture>
```

Counter-Q: You added `loading="lazy"` to every image and LCP got worse. Why?

A: Lazy-loaded images are excluded from the preload scanner's early discovery — the browser waits until layout determines the image is near the viewport, *then* starts the request. For the hero image that adds a full round trip **after** CSS and layout, directly inflating LCP. Rule: eager + `fetchpriority="high"` for anything in the initial viewport; lazy for everything below the fold, with a root margin of a few hundred pixels if you're using IntersectionObserver so images load just before they're scrolled into view.

Q16. How do you optimize web font loading?

Fonts are a classic CLS + LCP trap because they're discovered late (CSS must parse first) and block text rendering.

- **font-display**: `swap` shows fallback immediately then swaps (FOUT — good for LCP, risks CLS); `optional` uses the font only if it's ~instantly available (best for CLS, may not use your font on first visit); `block` hides text up to 3s (FOIT — avoid).
- **Self-host** in WOFF2 and `preload` the 1–2 critical weights with `crossorigin`. Google Fonts via CDN costs an extra origin connection plus a CSS round trip before the font is even discovered.
- **Subset** to the characters/scripts you need (`unicode-range`, `glyphhanger`, or subsetting variable fonts) — often cuts 70%+.
- **Variable fonts** replace 4–6 static weight files with one.
- **Match metrics** of the fallback to kill the swap shift: `size-adjust`, `ascent-override`, `descent-override` on a `@font-face` local fallback (or Next.js's automatic fallback adjustment).
- Limit families and weights — every extra weight is another file.

Counter-Q: font-display: swap vs optional — which do you pick and defend it.

A: It's an explicit LCP-vs-CLS trade. `swap` guarantees text is visible immediately (good LCP, good accessibility) but the swap causes a layout shift unless you've metric-matched the fallback. `optional` gives a ~100ms window; if the font isn't ready it renders the fallback for the whole page view and caches the font for *next* navigation — zero CLS, but first-time visitors may never see your brand font. My default: **swap + a metric-matched fallback via `size-adjust`**, which gets visible text fast and near-zero shift. Choose `optional` if brand typography is negotiable and CLS is a hard business constraint.

7. React-Specific Optimization

Q17. How do you optimize a slow React app?

Profile with React DevTools Profiler first — record an interaction, look at the flame graph for components rendering that shouldn't, and enable "Highlight updates."

- **Fix state placement:** the most common cause of over-rendering is state living too high. Move it down, or lift the expensive subtree out via `children` so it isn't re-created by the parent's render.
- `React.memo` on components that re-render with identical props (especially expensive leaves in a list).
- `useMemo` for genuinely expensive computation; `useCallback` for functions passed to memoized children or to dependency arrays.
- **Stabilize context:** a context value object recreated each render re-renders every consumer. Memoize it, and split one big context into several narrow ones (or use a store like Zustand/Jotai with selector subscriptions).
- **Virtualize** long lists (`@tanstack/react-virtual`, `react-window`).
- **Concurrent features:** `useTransition` to mark expensive updates non-urgent (keeps typing responsive), `useDeferredValue` to lag a derived heavy render behind fast input.
- **Keys:** stable, unique, not array index for reorderable lists — bad keys force unmount/remount of whole subtrees.
- **Lazy load** routes and heavy components; use `Suspense` with skeletons sized to prevent CLS.

Counter-Q: Should you just wrap everything in `useMemo` and `memo` to be safe?

A: No. Memoization isn't free: every `useMemo` / `useCallback` allocates, stores a dependency array, and runs a comparison on every render — for cheap computations that costs more than re-running them.

`React.memo` adds a props shallow-compare that is wasted work if props change every time anyway, and it silently does nothing when you pass an inline object/array/function. Blanket memoization also hurts readability and hides the *real* bug, which is usually bad state placement. Correct approach: profile, find the specific expensive render, fix the structure first, then memoize the hot path and verify the improvement. (Note: the React Compiler in React 19 automates most of this, which is further reason not to hand-scatter memos.)

Q18. Explain virtualization / windowing and implement the core idea.

Rendering 10,000 rows creates 10,000 DOM nodes → huge memory, slow initial layout, janky scroll. Virtualization renders only the ~20 rows in the viewport (plus an overscan buffer), absolutely positioned inside a spacer element sized to the full list height so the scrollbar stays accurate.

```
function VirtualList({ items, rowHeight = 40, height = 600 }) {
  const [scrollTop, setScrollTop] = useState(0);
  const overscan = 3;

  const start = Math.max(0, Math.floor(scrollTop / rowHeight) - overscan);
  const visible = Math.ceil(height / rowHeight) + overscan * 2;
  const end = Math.min(items.length, start + visible);

  return (
    <div style={{ height, overflow: 'auto' }} onScroll={e => setScrollTop(e.currentTarget.scrollY)}>
      <div style={{ height: items.length * rowHeight, position: 'relative' }}>
        {items.slice(start, end).map((item, i) => (
          <div key={item.id}
            style={{ position: 'absolute', top: (start + i) * rowHeight, height: rowHeight }}
            {item.label}
          </div>
        ))}
      </div>
    </div>
  );
}
```

Counter-Q: This assumes fixed row height. How do you handle variable-height rows?

A: You can't compute offsets arithmetically anymore. Standard solution: (1) **estimate** each row's height, (2) render, (3) **measure** actual heights with `ResizeObserver` or a ref callback, (4) cache measured heights in a **prefix-sum / Fenwick tree** so you can binary-search "which row is at scrollTop Y" in $O(\log n)$ and update a height in $O(\log n)$, (5) correct the scroll position when a measurement changes an offset above the viewport, otherwise the content visibly jumps. Libraries — `@tanstack/react-virtual` or `react-virtuoso` — implement exactly this plus sticky headers, reverse scrolling, and grid support. Also mention accessibility: virtualized lists break `Ctrl+F`, screen-reader item counts (fix with `aria-setsize / aria-posinset`), and anchor links — so don't virtualize a 200-row list where plain pagination or `content-visibility: auto` would do.

Q19. What causes unnecessary re-renders in React and how do you find them?

Causes:

- State updated in a common ancestor when only a leaf cares.
- New object/array/function identity passed as props each render (`style={{...}}` , `onClick={() => ...}`) defeating `memo` .
- Context value not memoized → all consumers re-render on any change.
- Index keys causing reconciliation to remount rows.
- Non-memoized selectors returning a fresh object (`useSelector(s => ({a: s.a}))`).
- Parent re-render cascading through non-memoized children.

How to find: React DevTools Profiler → "Record why each component rendered"; the "Highlight updates when components render" setting; `why-did-you-render` in dev; React Compiler's ESLint plugin for rule violations. Verify a fix by re-profiling — don't assume.

Counter-Q: Is a re-render always a performance problem?

A: No, and this is the most-common over-optimization in React. A re-render is React calling your function and diffing the returned element tree; for a small component that's microseconds and results in **zero DOM mutations** if nothing changed. DOM writes are the expensive part, and React already skips them. Re-renders matter when they're (a) frequent — per keystroke or per scroll frame, (b) wide — hundreds of components, or (c) expensive per component — heavy computation, large lists, chart re-layout. Optimize when the profiler shows a render exceeding your frame budget, not because the highlight flashed.

8. Memory, Long Lists & Runtime

Q20. What causes memory leaks in a SPA and how do you detect them?

Common causes:

- Event listeners on `window / document` never removed on unmount.
- `setInterval / setTimeout` not cleared.
- `IntersectionObserver / ResizeObserver / MutationObserver` not disconnected.
- Subscriptions (WebSocket, EventSource, store, RxJS) not unsubscribed.
- Closures capturing large objects, held alive by a long-lived callback.
- Detached DOM nodes still referenced from a JS array/map.
- Unbounded caches (a plain `Map` keyed by id that grows forever) — use `WeakMap / WeakRef` or an LRU with a cap.
- Stale state updates after unmount (also causes the classic React warning).

```
useEffect(() => {
  const controller = new AbortController();
  window.addEventListener('resize', onResize, { signal: controller.signal });
  const id = setInterval(poll, 5000);
  fetch(url, { signal: controller.signal });
  return () => { controller.abort(); clearInterval(id); }; // one signal cleans up all
}, []);
```

Detection: DevTools Memory → take a heap snapshot, perform the suspect action N times, snapshot again, and use **"Comparison"** view sorted by delta; filter by "Detached" to find detached DOM. The **Allocation instrumentation on timeline** shows what's allocated and never freed. A sawtooth-that-never-returns-to-baseline in the Performance monitor's JS heap is the signature.

Counter-Q: Memory usage grows during a session but the app never crashes. Is that actually a leak?

A: Not necessarily. Growth can be legitimate caching (query cache, images, virtualized item measurements) and V8's GC is lazy — it won't collect until pressure demands it, so a rising graph alone proves nothing. The test is: force GC (the trash icon in the Memory panel), return the app to a known-equivalent state, and check whether the heap returns to baseline. Repeat the cycle 3–5 times; a real leak shows **monotonic growth across cycles**. Also weigh impact — a slow leak that adds 2MB per hour on a page users keep open for 5 minutes is not worth fixing; the same leak in a trading dashboard open for 8 hours absolutely is. On mobile, memory pressure means the OS silently kills your tab, which users experience as "the app reloads and loses my work."

9. Rendering Architecture (CSR / SSR / SSG / ISR)

Q21. Compare CSR, SSR, SSG, and ISR and pick one for a given product.

Strategy	How	Pros	Cons	Fits
CSR	Empty HTML shell, JS renders in browser	Cheap hosting, rich interactivity, simple	Slow FCP/LCP, poor SEO, big JS cost on low-end devices	Dashboards, internal tools, behind-login apps
SSR	HTML rendered per request on the server	Fast FCP, SEO, always-fresh personalized data	Server cost/latency per request, TTFB depends on backend, hydration cost	E-commerce PDPs, social feeds, anything personalized + SEO
SSG	HTML built at deploy time, served from CDN	Fastest possible TTFB, trivially scalable, cheap	Rebuild for content changes; impractical at 100k+ pages	Docs, marketing, blogs
ISR	SSG + background regeneration on a revalidate interval / on-demand	CDN speed with fresh-ish content, no full rebuilds	Users may see stale content briefly; framework-specific	Large catalogs, news, listings

Modern default in React land: **RSC + streaming SSR** — server components ship zero JS for non-interactive parts, and `<Suspense>` streams slow sections in later so the shell paints immediately.

Counter-Q: SSR made FCP fast but INP got worse. Explain and fix.

A: That's the **hydration gap** (the "uncanny valley"): the server HTML paints quickly so the page *looks* ready, but it's inert until the client bundle downloads, parses, and walks the whole tree attaching listeners and rebuilding state. Hydration is one big long task — clicks during it are queued or dropped, which is exactly what INP measures. Fixes: (1) **selective/progressive hydration** — `<Suspense>` boundaries so React hydrates in chunks and prioritizes the part the user interacted with; (2) **React Server Components** so non-interactive subtrees never hydrate at all; (3) **islands architecture** (Astro/Qwik-style) — ship JS only for interactive islands; Qwik goes further with resumability, skipping hydration entirely; (4) shrink the client bundle, since hydration cost scales with component count and bundle size; (5) don't render interactive-looking controls before they work — disable or show a loading affordance.

Q22. How do you optimize third-party scripts you don't control?

Third parties routinely account for the majority of main-thread blocking. Approach:

1. **Audit & justify:** attribute each script to a business owner and a measured value. The cheapest optimization is deletion — many tags are from campaigns that ended years ago.
2. **Load late:** `async / defer`, or load on interaction / on idle / when scrolled into view (chat widgets, video embeds — use a facade like `lite-youtube-embed` that swaps in the real iframe on click).
3. **Isolate:** Partytown runs third-party scripts inside a Web Worker with a proxied DOM — good for analytics/tag managers, not for anything needing synchronous DOM.
4. **Constrain:** `<iframe>` sandboxing for ads; reserve their space with fixed dimensions to prevent CLS.
5. **Connect early:** `preconnect` to their origin so DNS/TLS isn't on the critical path.
6. **Self-host** where licensing permits (e.g. GA's `gtag.js`) to remove an origin and gain cache control.
7. **Guard:** set a Lighthouse CI budget on third-party bytes and alert on regressions — tag managers let non-engineers ship code to production.

Counter-Q: Marketing insists the tag manager must load synchronously in the head. What do you do?

A: Turn it into a data conversation rather than a veto. Quantify: measure TBT/INP and conversion with and without, ideally via an A/B test — "this tag costs 400ms of blocking time; our data says every 100ms costs ~1% conversion." Then offer a middle path: keep only the genuinely needed synchronous piece (usually just the consent/anti-flicker snippet, which can be a few inlined lines) and defer the rest; move analytics into Partytown; or use server-side tagging so the browser does none of the work. Also set a **governance policy**: new tags require review and a size budget. Escalate with numbers, not opinions — and if the business accepts the cost after seeing the data, that's a legitimate decision, and you make sure it's monitored.

10. Rapid-Fire Round

Q23. Short answers you should have ready

Question	Answer
CSS in head, JS at end of body — still true?	CSS in head, yes (render-blocking, avoids FOUC). JS at body-end is superseded by <code>defer</code> in head — same non-blocking behavior but the script is <i>discovered and downloaded earlier</i> .
Gzip vs Brotli	Brotli ~15–25% smaller on text at comparable speed; use level 11 for static (precompressed at build), 4–5 for dynamic. Keep gzip as fallback.
What is TTI and why did it fall out of favor?	Time To Interactive — first 5s window with no long tasks. Noisy and easily gamed; replaced in practice by TBT (lab) and INP (field).
Reduce CLS quickly	Set explicit dimensions/aspect-ratio on media, reserve space for ads/embeds/banners, avoid inserting content above existing content, use <code>transform</code> for animation instead of properties that shift layout, metric-match font fallbacks.
<code>content-visibility: auto</code>	Tells the browser to skip rendering (layout+paint) of off-screen subtrees. Pair with <code>contain-intrinsic-size</code> so the scrollbar doesn't jump. Huge win on long documents, near-free.
Critical CSS	Inline the above-the-fold CSS in <code><head></code> (<14KB ideally) and load the rest async via <code><link rel="preload" as="style" onload="this.rel='stylesheet'"></code> . Removes a render-blocking round trip.
Reduce API latency perception	Optimistic UI, skeletons sized to final content, request deduping + caching (React Query/SWR), pagination/cursoring, parallel not waterfalled requests, <code>AbortController</code> to cancel stale ones, prefetch on hover.
Why is <code>key={index}</code> bad?	On reorder/insert, React matches elements by key, so index keys pair the wrong old element with the new one — causing wrong state on inputs, lost DOM identity, unnecessary re-renders, and broken animations.
Biggest perf win in most real apps	Ship less JavaScript. Then: optimize images, then cache aggressively, then fix the main thread. In that order.

Counter-Q: If you could make only one change to a slow app, what would it be — and how would you justify it?

A: I wouldn't pick a change first — I'd pick a **measurement**. Without knowing whether the bottleneck is TTFB, payload, or main thread, any single change is a guess with a ~1-in-3 chance. Set up RUM, segment by route and device, find where the p75 actually fails, and let that dictate the change. The strongest answer to any "optimize X" question is: measure, form a hypothesis, change one thing, re-measure, and tie the result to a business metric. Performance work without numbers is folklore.